Containerizing our SpringBoot Application

1. Goal

- The objective is to **Dockerize/containerize** our SpringBoot Application.
- We will convert the SpringBoot application into a **Docker Image,** which can then be used to create a container.

2. Where to Run the Docker Command

- Use a terminal available on your machine.
- The terminal can be:
  - Windows terminal
  - macOS/Linux Terminal
  - IntelliJ’s built-in terminal
- Make sure you are in the **root directory** of the **SpringBoot Project.**

In IntelliJ:

```
IntelliJ
   ↓
Terminal
   ↓
Spring Boot project root directory
```

3. Creating the Docker Image

- The lecture uses Maven Wrapper to create the Docker Image.
- The command is based on `./mvnw spring-boot:build-image`
- if maven is installed **globally** then we can use: `mvn spring-boot:build-image`
- Maven Wrapper
  - **mvnw – Maven Wrapper**
  - It is used to execute Maven commands
  - It helps ensure the required Maven setup is available for the project.

4. Spring-Boot:build-image

- This is a **Maven goal** provided by **SpringBoot**
- It tells **SpringBoot** to **automatically build** a **Docker-compatible image**
- With this approach:
  - We don’t need to manually create a Dockerfile.
  - SpringBoot automatically determines what is needed for the application image
  - The required dependencies and configuration are handled automatically

5. Giving the Image a Name

- The command also specifies the **Docker Image Name.**
- The image name should generally include your Docker Hub username and application name.
- Example : username/application-name
- Replace username with your own **Docker Hub username** and application-name with our application-name

6. Docker Hub Account

- To push the image to Docker Hub, we need a Docker Hub Account.
- We can
  - Signup for a new account
  - Signin to an existing account
- Docker Hub supports authentication options such as:
  - Email/password
  - Google
  - GitHub

7. Docker Desktop must be running

- Before executing the image-building command **Docker Desktop must be running**
- Docker Desktop starts the Docker Engine
- If Docker isn’t running, the Docker command will fail.

```
Docker Desktop
      ↓
Docker Engine
      ↓
Ready to build image
```

8. What Happens during the build?

- SpringBoot application is started/built as part of the process.
- The required **Paketo buildpacks** are pulled
- Buildpacks analyze/package the application
- Different layers are created
- These layers together form the **Docker Image**
- The image is **saved locally**
- The build finishes successfully

When the command runs:

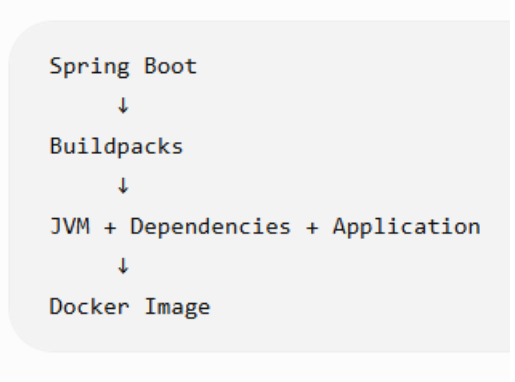
```
mvnw spring-boot:build-image
```

the process is roughly:

```
Spring Boot Application
        ↓
Maven Wrapper
        ↓
Spring Boot Maven Plugin
        ↓
Build the application
        ↓
Paketo Buildpacks
        ↓
Create layers
        ↓
Docker Image
```

9. Paketo Buildpacks

- The build process pulls the **Paketo Buildpacks.**
- Buildpacks handle much of the work that would otherwise be specified manually in a Dockerfile.
- They determine what is required to package the SpringBoot application into a container image.



10. Layers

- Buildpacks create separate **layers** within the image.
- These layers can contain different parts of the application such as
  - JVM/Runtime
  - Dependencies
  - Application
- Docker can cache these layers
- If you make another build after a small code change:
  - Unchanged layers can be re-used
  - Only changed parts need to be rebuilt.
- This improves build performance.

11. Verify the Image in Docker Desktop

- After the build succeeds
  - Open **Docker Desktop**
  - Go to the **Images** section
  - Our newly created application image is available

Example:

```
Docker Desktop
     ↓
Images
     ↓
ecom-application
```

At this point:

> **The image exists locally on your computer.**

12. Push the Image to Docker Hub

- Once the image is available locally, the next step is to push it to Docker Hub
- But first we need to authenticate
- In terminal use `docker login`
- Docker may provide browser-based authentication.
- It can give us a URL and a confirmation code.
- Complete the authentication in your browser
- Once successful, the command-line will show `Login Succeeded`

Alternative Authentication

- We can also authenticate using username/credentials through the command line.

```
Browser-based login
        OR
Command-line authentication
```

13. Docker Push

- After authentication, push the image:

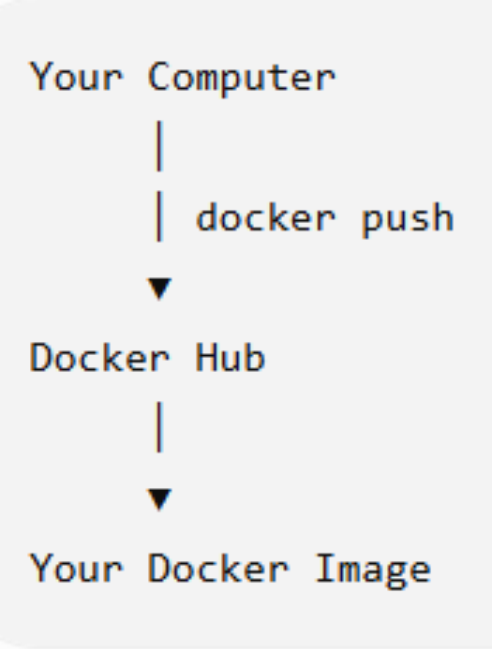
```
docker push username/ecom-application
```

- Docker uploads the image to **Docker Hub**
- The layers are pushed to the remote registry
- The process depends on
  - Internet Speed
  - System Performance
  - Image Size

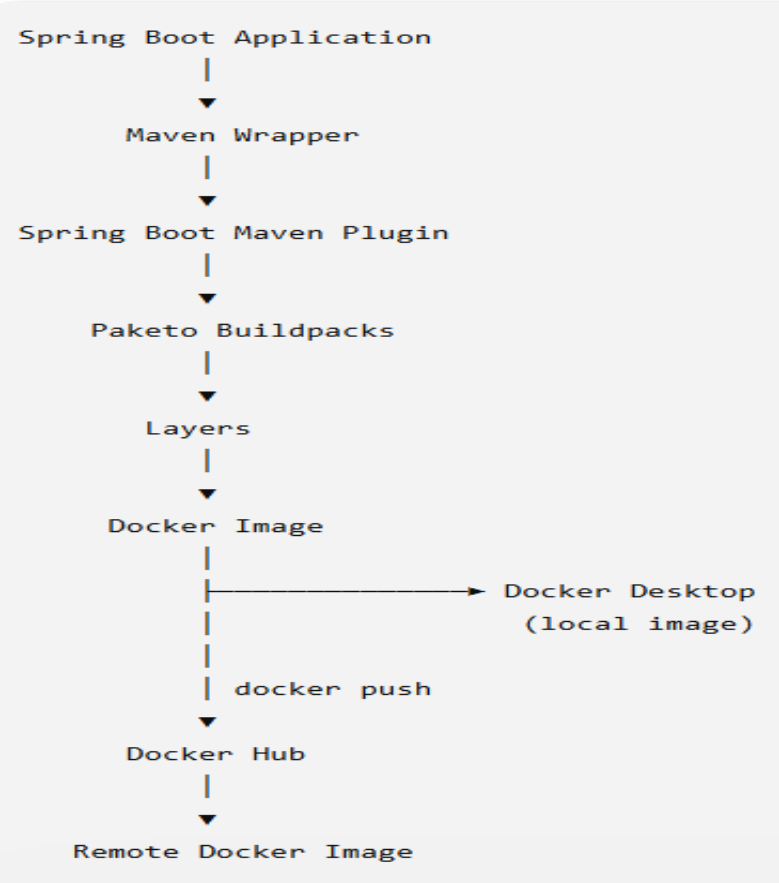
14. Verify on Docker Hub

- After the push completes:

1. Open Docker Hub
2. Refresh your account/repository page
3. Find your application image
4. Open the image repository
5. Now, Docker Image is available remotely



15. Complete Flow



16. Commands used

Bash

```
# Build Docker image
mvnw spring-boot:build-image
```

Bash

```
# Login
docker login
```

Bash

```
# Push image
docker push username/application-name
```